Module 1: Foundations

Introduction to Relational Databases | History of PostgreSQL | Features of PostgreSQL

Datapundit — PostgreSQL DBA Training Series

Part A: Introduction to Relational Databases

A.1 What is a Database?

A **database** is an organized collection of structured data stored electronically, designed for efficient storage, retrieval, and management. A **DBMS (Database Management System)** is the software layer that lets applications and users create, read, update, and delete that data without dealing with the physical storage details directly.

A.2 Types of Database Systems (Brief Landscape)

Type	Model	Examples
Hierarchical	Tree-structured, parent-child	IMS
Network	Graph of records	IDMS
Relational (RDBMS)	Tables with rows/columns, set theory based	PostgreSQL, MySQL, Oracle, SQL Server
NoSQL	Document, key-value, column, graph	MongoDB, Redis, Cassandra, Neo4j
NewSQL	Distributed + SQL + ACID	CockroachDB, YugabyteDB, TiDB

Trainer note: Since your audience will likely include people transitioning from MongoDB/MySQL, this is a good place to briefly contrast the relational model's fixed schema and join-based querying against the document model's flexible schema.

A.3 The Relational Model

Introduced by **Dr. Edgar F. Codd** in his 1970 paper *"A Relational Model of Data for Large Shared Data Banks"* while at IBM. Core idea: represent all data as **relations (tables)** — mathematically grounded in set theory and predicate logic, rather than navigational pointers between records (as in hierarchical/network models).

Terminology mapping:

Formal Term	Common Term
Relation	Table
Tuple	Row / Record
Attribute	Column / Field
Domain	Data type / valid value set
Cardinality	Number of rows
Degree	Number of columns

```sql
-- A "relation" (table) named employees, with "attributes" as columns
CREATE TABLE employees (
    employee_id   SERIAL PRIMARY KEY,
    first_name    VARCHAR(50),
    last_name     VARCHAR(50),
    department_id INT
);

-- Each row inserted below is a "tuple"
INSERT INTO employees (first_name, last_name, department_id)
VALUES ('Anitha', 'Rao', 10);
```

A.4 Keys and Relationships

- **Primary Key (PK):** Uniquely identifies each row; cannot be NULL.
- **Candidate Key:** Any column (or set) that could qualify as PK.
- **Composite Key:** PK made of two or more columns.
- **Foreign Key (FK):** Column referencing a PK in another table — enforces referential integrity.
- **Unique Key:** Enforces uniqueness but allows one NULL (unlike PK).

Relationship types:

- One-to-One (1:1) — e.g., `employee` ↔ `employee_passport_details`
- One-to-Many (1:N) — e.g., `department` ↔ `employees`
- Many-to-Many (M:N) — resolved via a junction/bridge table, e.g., `students` ↔ `courses` via `enrollments`

```sql
-- Primary Key
CREATE TABLE department (
    department_id   SERIAL PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL
);

-- Foreign Key referencing department (1:N relationship)
CREATE TABLE employees (
    employee_id   SERIAL PRIMARY KEY,
    full_name     VARCHAR(100),
    email         VARCHAR(100),
    department_id INT REFERENCES department(department_id)
);

-- Unique Key (allows one NULL, unlike PK)
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);

-- Composite Primary Key (junction table resolving an M:N relationship)
CREATE TABLE enrollments (
    student_id  INT REFERENCES students(student_id),
    course_id   INT REFERENCES courses(course_id),
    enrolled_on DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, course_id)
);
```

A.5 SQL — The Language of RDBMS

SQL (Structured Query Language) is divided into sub-languages:

Category	Purpose	Commands
DDL	Data Definition	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DML	Data Manipulation	`INSERT`, `UPDATE`, `DELETE`
DQL	Data Query	`SELECT`
DCL	Data Control	`GRANT`, `REVOKE`
TCL	Transaction Control	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

```sql
```

```sql
-- DDL: define/alter structure
CREATE TABLE products (product_id SERIAL PRIMARY KEY, name VARCHAR(100));
ALTER TABLE products ADD COLUMN price NUMERIC(10,2);
DROP TABLE IF EXISTS temp_products;

-- DML: manipulate data
INSERT INTO products (name, price) VALUES ('Areca Nut - 1kg', 250.00);
UPDATE products SET price = 275.00 WHERE product_id = 1;
DELETE FROM products WHERE product_id = 99;

-- DQL: query data
SELECT product_id, name, price
FROM products
WHERE price > 100
ORDER BY price DESC;

-- DCL: control access
GRANT SELECT, INSERT ON products TO app_readwrite;
REVOKE INSERT ON products FROM app_readwrite;

-- TCL: control transactions
BEGIN;
UPDATE products SET price = price * 1.05;
SAVEPOINT before_discount;
UPDATE products SET price = price * 0.9 WHERE product_id = 1;
ROLLBACK TO before_discount;
COMMIT;
```

A.6 ACID Properties

This is the single most important concept to anchor before going further — everything from WAL to MVCC to isolation levels traces back here.

- **Atomicity** — A transaction is all-or-nothing; partial writes never persist.
- **Consistency** — A transaction moves the database from one valid state to another, respecting constraints.
- **Isolation** — Concurrent transactions don't see each other's uncommitted changes (governed by isolation levels: Read Committed, Repeatable Read, Serializable).
- **Durability** — Once committed, data survives crashes (this is where WAL comes in, covered in Module 2).

```sql
```

```sql
-- Atomicity: a funds transfer either fully happens or not at all
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 'A1';
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 'A2';
COMMIT;   -- both updates persist together; a crash before COMMIT loses neither partial

-- Isolation: explicitly setting an isolation level for a transaction
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE account_id = 'A1';
-- ... application logic using that snapshot ...
COMMIT;
```

A.7 Normalization (Brief)

Process of organizing columns/tables to minimize redundancy and avoid update/insert/delete anomalies.

Normal Form	Rule
1NF	Atomic values, no repeating groups
2NF	1NF + no partial dependency on composite key
3NF	2NF + no transitive dependency
BCNF	Stricter version of 3NF

sql

```sql
-- Un-normalized design (1NF violation): repeating values crammed into one column
-- orders(order_id, customer_name, items)
-- e.g. items = 'Pen, Notebook, Eraser'  -- hard to query, update, or constrain

-- After normalizing to 3NF:
CREATE TABLE customers (
    customer_id   SERIAL PRIMARY KEY,
    customer_name VARCHAR(100)
);

CREATE TABLE orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date  DATE
);

CREATE TABLE order_items (
    order_id  INT REFERENCES orders(order_id),
    item_name VARCHAR(100),
    quantity  INT,
    PRIMARY KEY (order_id, item_name)
);
```

Most production OLTP schemas target **3NF**; OLAP/reporting schemas often deliberately denormalize (star/snowflake schemas) for query performance.

A.8 Why Relational Databases Still Matter

- Strong consistency guarantees (ACID) — critical for financial, inventory, and transactional systems
- Mature tooling, decades of optimizer research, predictable query planning
- Declarative querying (SQL) vs. imperative data access
- Rich ecosystem: replication, backup, monitoring, compliance tooling

---

Part B: History of PostgreSQL

B.1 Origins — Ingres (1970s, UC Berkeley)

PostgreSQL's lineage starts with **Ingres**, a relational database project led by **Michael Stonebraker** at UC Berkeley in the 1970s — one of the earliest implementations of Codd's relational model, alongside IBM's System R.

B.2 The POSTGRES Project (1986–1994)

Stonebraker returned to Berkeley to start a successor project literally named **POSTGRES** ("post-Ingres"), aiming to address limitations of relational systems at the time — notably support for complex data types and user-defined types/functions, which was ahead of its time. The project ran from 1986 to 1994 and used a query language called **QUEL**, not SQL.

B.3 Postgres95 (1995)

Two Berkeley students, **Andrew Yu and Jolly Chen**, replaced QUEL with a SQL interpreter, and the project was released as open source under the name **Postgres95**.

B.4 PostgreSQL (1996 onward)

In 1996, the project was renamed **PostgreSQL** to reflect SQL support, and an outside group of developers took over from the university, forming what's now the **PostgreSQL Global Development Group (PGDG)** — a distributed, volunteer-driven community with no single corporate owner. Version numbering restarted at **6.0** in 1997.

```sql
-- A nod to the QUEL-to-SQL transition: every modern install speaks SQL,
-- and you can always confirm exactly which PostgreSQL version you're on with:
SELECT version();
```

B.5 Major Milestones Timeline

Year	Version	Key Addition
1996	Postgres95 → PostgreSQL	SQL support, renamed
1997	6.0	First PGDG community release
2005	8.0	Native Windows support, savepoints, tablespaces
2010	9.0	Built-in streaming replication, hot standby
2012	9.2	Native JSON support
2014	9.4	**JSONB** (binary, indexable JSON)
2016	9.6	Parallel query execution
2017	10	Native logical replication, declarative partitioning
2018	11	Partitioning improvements, JIT compilation, stored procedures
2020	13	B-tree index dedup, parallel vacuum
2022	15	`MERGE` command, better logical replication
2023	16	Logical replication from standbys, parallelism improvements
2024	17	Improved vacuum memory management, incremental backup (`pg_basebackup`)
2025	18	Async I/O subsystem, further vacuum/observability work
2026	19 (Beta)	Vacuum, logical decoding, and temporal table improvements — GA expected ~Sep/Oct 2026

B.6 Governance & Licensing

- **Governance:** No single company controls PostgreSQL. Development is driven by the PGDG — a global community of contributors, with a Core Team providing coordination. Companies like EDB, Crunchy Data, Percona, and Microsoft (via citus/flexible server) contribute code but don't own the project.
- **License:** The **PostgreSQL License** — a permissive, OSI-approved license similar to MIT/BSD. No restrictions on commercial use, forking, or redistribution, and no copyleft obligations (unlike GPL).

- **Release cadence:** One major version per year (typically September/October), with quarterly minor/patch releases. Each major version gets **5 years of support.**

Part C: Features of PostgreSQL

C.1 Standards Compliance & Extensibility

- Strong SQL standard compliance (SQL:2016 and beyond features like window functions, CTEs, `MERGE`)
- **Extensibility is PostgreSQL's defining trait**, inherited directly from the original POSTGRES research goals: custom data types, operators, functions, index types, and full extensions (`CREATE EXTENSION`)
- Popular extensions: `PostGIS` (spatial), `pg_stat_statements` (query stats), `pgcrypto`, `pg_cron`, `TimescaleDB`, `Citus` (distributed)

```sql
-- Enabling extensions
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Extensibility in action: a user-defined function
CREATE OR REPLACE FUNCTION full_name(first_name TEXT, last_name TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN first_name || ' ' || last_name;
END;
$$ LANGUAGE plpgsql;

SELECT full_name('Shri', 'Rao');
```

C.2 Concurrency: MVCC

PostgreSQL uses **Multi-Version Concurrency Control** — readers never block writers and writers never block readers, because each transaction sees a consistent snapshot. This is also *why* VACUUM exists (dead tuple cleanup) — a natural bridge into Module 3.

```sql
-- MVCC bookkeeping is visible on every table via hidden system columns
SELECT xmin, xmax, ctid, employee_id, first_name
FROM employees
WHERE employee_id = 1;
```

C.3 Advanced Data Types

- `JSON` / `JSONB` (binary, indexable — a major differentiator vs. traditional RDBMS)
- Arrays (native, any base type)
- `hstore` (key-value pairs)
- Range types (`int4range`, `tsrange`, etc.)
- Geometric types (`point`, `polygon`, `circle`)
- Network types (`inet`, `cidr`, `macaddr`)
- `UUID`, `XML`, composite types, enum types

sql

```sql
-- JSONB
CREATE TABLE products (
    id         SERIAL PRIMARY KEY,
    name       TEXT,
    attributes JSONB
);
INSERT INTO products (name, attributes)
VALUES ('Coconut Oil 1L', '{"organic": true, "origin": "Kinnigoli"}');

SELECT name FROM products WHERE attributes->>'organic' = 'true';

-- Array type
CREATE TABLE farms (id SERIAL PRIMARY KEY, crops TEXT[]);
INSERT INTO farms (crops) VALUES (ARRAY['coconut', 'areca nut']);
SELECT * FROM farms WHERE 'areca nut' = ANY(crops);

-- Range type
CREATE TABLE bookings (id SERIAL PRIMARY KEY, stay tsrange);
INSERT INTO bookings (stay) VALUES ('[2026-08-01, 2026-08-05)');
SELECT * FROM bookings WHERE stay @> '2026-08-03'::timestamp;
```

C.4 Indexing Options

Index Type	Best For
B-tree	Default, equality/range queries
Hash	Equality-only lookups
GiST	Geometric, full-text, nearest-neighbor
SP-GiST	Non-balanced trees (e.g., IP ranges, quad-trees)
GIN	Full-text search, JSONB, arrays
BRIN	Very large, naturally ordered tables (block range)

```sql
CREATE INDEX idx_employees_lastname   ON employees USING btree (last_name);
CREATE INDEX idx_employees_email_hash ON employees USING hash  (email);
CREATE INDEX idx_products_attrs_gin   ON products  USING gin   (attributes);
CREATE INDEX idx_bookings_stay_gist   ON bookings  USING gist  (stay);
CREATE INDEX idx_logs_created_brin    ON logs      USING brin  (created_at);
```

C.5 Replication & High Availability

- **Physical (streaming) replication** — byte-level WAL shipping, since 9.0
- **Logical replication** — table/row-level, selective, cross-version since PG10
- Synchronous and asynchronous modes
- Ecosystem tools: Patroni, repmgr, pgBackRest, Barman

```sql
-- On the primary/publisher
CREATE PUBLICATION sales_pub FOR TABLE orders, order_items;

-- On the subscriber
CREATE SUBSCRIPTION sales_sub
    CONNECTION 'host=primary_host dbname=salesdb user=replicator password=xxxx'
    PUBLICATION sales_pub;
```

C.6 Partitioning

Declarative partitioning (range, list, hash) since PG10, with steady maturity improvements every release since (partition pruning, foreign key support across partitions, etc.)

```sql
CREATE TABLE sales (
    sale_id   BIGSERIAL,
    sale_date DATE NOT NULL,
    amount    NUMERIC(12,2)
) PARTITION BY RANGE (sale_date);

CREATE TABLE sales_2026_q1 PARTITION OF sales
    FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');

CREATE TABLE sales_2026_q2 PARTITION OF sales
    FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
```

C.7 Procedural Languages

Native support for stored procedures/functions in multiple languages: `PL/pgSQL` (default), `PL/Python`, `PL/Perl`, `PL/Tcl`, and more via extensions — a flexibility most RDBMS don't offer out of the box.

```sql
CREATE OR REPLACE PROCEDURE apply_discount(p_product_id INT, p_pct NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE products
    SET price = price - (price * p_pct / 100)
    WHERE id = p_product_id;
END;
$$;

CALL apply_discount(1, 10);
```

C.8 Security

- Role-based access control (roles can be users and/or groups)
- **Row-Level Security (RLS)** — policy-based row filtering per role
- SSL/TLS for connections, `pgcrypto` for column-level encryption
- Fine-grained `GRANT`/`REVOKE` privilege system

```sql
```

```sql
-- Roles & grants
CREATE ROLE app_readonly LOGIN PASSWORD 'xxxx';
GRANT CONNECT ON DATABASE salesdb TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;

-- Row-Level Security
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY region_isolation ON orders
    USING (region = current_setting('app.current_region'));
```

C.9 Performance Features

- Parallel query execution (seq scans, joins, aggregates)
- JIT (Just-In-Time) compilation of expressions, since PG11
- Sophisticated cost-based query planner/optimizer
- Foreign Data Wrappers (`postgres_fdw`, and dozens of community FDWs) for querying external systems as if local

```sql
EXPLAIN ANALYZE
SELECT customer_id, SUM(amount)
FROM orders
WHERE order_date >= '2026-01-01'
GROUP BY customer_id
ORDER BY SUM(amount) DESC;
```

C.10 Why This Matters for a DBA

Every feature above becomes a training module later in this series: MVCC → VACUUM, WAL → backup/recovery, replication → HA design, indexing → performance tuning. Part C is essentially your table of contents in disguise — worth calling out explicitly to trainees so they see where the course is headed.

Discussion Questions / Exercises for Trainees

1. Explain in your own words why ACID matters more for a banking application than for a social media "like" counter.
2. Draw an ER diagram for a simple `students`–`courses` many-to-many relationship, including the junction table.
3. Why do you think PostgreSQL chose a permissive license instead of GPL? What tradeoffs does that involve for the project's growth?

4. Name three PostgreSQL features that a traditional RDBMS (e.g., older MySQL versions) historically lacked, and explain why each matters operationally.
5. Write a `CREATE TABLE` statement for an `order_items` junction table with a composite primary key, then write an `INSERT` and a `SELECT` that uses it.

Key Takeaways

- The relational model (Codd, 1970) organizes data as tables governed by keys and constraints, queried via SQL.
- ACID properties are the contract a DBMS makes with its data — durability (via WAL) and isolation (via MVCC) are where PostgreSQL internals will matter most later in this course.
- PostgreSQL traces a 45+ year lineage from Ingres → POSTGRES → Postgres95 → PostgreSQL, and remains community-governed under a permissive license.
- PostgreSQL's defining trait is **extensibility** — most "advanced" features (JSONB, custom types, FDWs, procedural languages) trace back to that original design philosophy.